EVerest Integration

Application Note

Contents

Overview 3
Quick Start 3
Setup 3
Run DC EV Simulation 4
Run DC EVSE 4
EVerest Modules 4
Raspi EVSE Hardware Module 4
Detailed Description 4
Module Configuration 5
Raspi EV Hardware Module 5
Detailed Description 5
Module Configuration 5
EVerest Configuration 5
Example DC EVSE configuration 5
Example DC EV simulation configuration 8

Overview

This application note describes how to use the Raspi HAT with the EVerest project to implement basic EVSE and EV simulations. The instructions are based on EVerest release 2026.02.1 and hardware module implementation and configuration in https://github.com/komakiou/raspi-evse-everest-pub

According to the official EVerest documentation, EVerest is:

- a full-stack environment for EV-charging,
- a Linux Foundation Energy project,
- open-source,
- modular and customizable,
- licensed under the Apache 2.0 license to allow commercial-friendly use and
- backed by strong industry partners via great compatibility

For a comprehensive understanding of EVerest, including development, configuration, and deployment, refer to the official EVerest documentation:
https://everest.github.io/latest/index.html

Quick Start

Clone the raspi-evse-everest-pub repository onto the Raspberry Pi. For convenience, this repository already includes a copy of EVerest release 2026.02.01. Compiling EVerest can be time-consuming. For development and testing of EVerest modules, a Raspberry Pi with at least 4 GB of RAM is strongly recommended.

A faster model will significantly reduce build times. Even on the Raspberry Pi described above, running from a microSD card, a clean build can take more than one hour.

Setup

This setup basically follows EVerest documentation:

1. Clone the git repository that includes EVerest release 2026.02.1, hardware drivers and configuration.
```
$ git clone https://github.com/komakiou/raspi-evse-everest-pub
```

2. Install needed dependencies
```
$ sudo apt install -y python3-pip python3-venv git rsync wget cmake doxygen
graphviz build-essential clang-tidy cppcheck openjdk-21-jdk npm docker.io
docker-compose libboost-all-dev nodejs libssl-dev libsqlite3-dev   clang-
format curl rfkill libpcap-dev libevent-dev pkg-config libcap-dev
libsystemd-dev
```

3. Build the complete project
```
$ mkdir build && cd build
$ cmake ../EVerest-2026.02.1/
```

```
$ make -j2
$ make install
$ make iso15118_pip_install_dist
```

Run DC EV Simulation

In order to run the EV simulation execute the following commands in the build folder:

```
$ source venv/bin/activate
$ chmod +x run-scripts/run-raspi-ev-DC.sh
$ ./run-scripts/run-raspi-ev-DC.sh
```

Run DC EVSE

In order to run the DC EVSE simulation execute the following commands in the build folder:

```
$ chmod +x run-scripts/run-raspi-evse-DC.sh
$ ./run-scripts/run-raspi-evse-DC.sh
```

EVerest Modules

EVerest consists of multiple modules, each responsible for a specific part of the EV charging stack. These modules communicate with each other through MQTT messages. EVerest defines standardized interfaces and data types to ensure consistent communication between modules.

Modules responsible for interfacing with physical hardware are located in **`EVerest/modules/HardwareDrivers.`**

Raspi EVSE Hardware Module

This module provides the hardware interface between the Raspi EV(SE) HAT, configured in EVSE mode, and other EVerest modules.

The module is located in:
`EVerest-2026.02.1/modules/HardwareDrivers/EVSE/RaspiEvseBSP`

When combined with other modules included in EVerest, it can be used to build either an AC or DC charging station. The EVerest configuration section demonstrates how this module is integrated into a basic DC charging station simulation.

Detailed Description

The module implements the following interfaces:

- **evse_board_support**: Handles I^2C communication with the HAT's CP controller to access and configure EVSE-related parameters.
- **powermeter**: Provides a basic simulation of a power meter, generating current and voltage measurements.
- **connector_lock**: Provides a skeleton implementation for a connector locking mechanism.

- **rcd**: Provides a skeleton implementation for a residual current device (RCD).

Module Configuration

The module's provided interfaces and configuration parameters are defined in its manifest.yaml file, located at:
`EVerest-2026.02.1/modules/HardwareDrivers/EVSE/RaspiEvseBSP/manifest.yaml`

Raspi EV Hardware Module

This module provides access to the EV-related features of the Raspi EV(SE) HAT. EVerest also includes modules that provide basic EV simulation functionality for testing a charging station.

The configuration section provides an example of how to combine these modules with the Raspi EV module to create a basic EV simulation.

Detailed Description

This module implements the following interface:

- **ev_board_support**: Handles I^2C communication with the HAT's CP controller to access and configure EV-related parameters.

Module Configuration

The module's provided interfaces and configuration parameters are defined in its manifest.yaml file, located at:
`EVerest-2026.02.1/modules/HardwareDrivers/EV/RaspiEvBSP/manifest.yaml`

EVerest Configuration

To combine multiple modules into a charging station or EV simulation, a configuration file must be created. It defines the modules to be used, their configuration parameters, and how they are connected to each other.

Example DC EVSE configuration

The file **`EVerest-2026.02.1/config/config-raspi-evse-DC.yaml`** defines a basic DC charging station simulation using the Raspi EV(SE) HAT:

```
active_modules:
  RaspiEvseBSP0:
    module: RaspiEvseBSP
    connections: {}
    config_module:
      i2c_dev: /dev/i2c-1
      i2c_freq: 10000
      i2c_addr: 96
      caps_min_current_A: 0
      caps_max_current_A: 10
  EvseManager0:
    module: EvseManager
    connections:
      bsp:
```

```
        - module_id: RaspiEvseBSP0
          implementation_id: board_support
      powermeter_car_side:
        - module_id: RaspiEvseBSP0
          implementation_id: powermeter
      slac:
        - module_id: EvseSlac0
          implementation_id: main
      hlc:
        - module_id: EvseV2G0
          implementation_id: charger
      powersupply_DC:
        - module_id: DCSupplySimulator0
          implementation_id: main
      powermeter_grid_side:
        - module_id: DCSupplySimulator0
          implementation_id: powermeter
      imd:
        - module_id: IMDSimulator0
          implementation_id: main
    config_module:
      ac_enforce_hlc: false
      ac_hlc_enabled: false
      ac_hlc_use_5percent: false
      ac_nominal_voltage: 230
      ac_with_soc: false
      autocharge_use_slac_instead_of_hlc: false
      cable_check_enable_imd_self_test: false
      cable_check_wait_below_60V_before_finish: false
      cable_check_wait_number_of_imd_measurements: 1
      charge_mode: DC
      connector_id: 1
      connector_type: cCCS2
      dbg_hlc_auth_after_tstep: false
      dc_isolation_voltage_V: 0
      disable_authentication: true
      enable_autocharge: false
      ev_receipt_required: false
      evse_id: DE*PNX*E1234567*1
      evse_id_din: 49A80737A45678
      external_ready_to_start_charging: false
      fail_on_powermeter_errors: false
      hack_allow_bpt_with_iso2: false
      hack_fix_hlc_integer_current_requests: false
      hack_pause_imd_during_precharge: false
      hack_present_current_offset: 0
      hack_simplified_mode_limit_10A: false
      hack_skoda_enyaq: false
      has_ventilation: false
      initial_meter_value_timeout_ms: 5000
      lock_connector_in_state_b: false
      logfile_suffix: session_uuid
      payment_enable_contract: false
      payment_enable_eim: true
      raise_mrec9: false
      request_zero_power_in_idle: false
      sae_j2847_2_bpt_enabled: false
      sae_j2847_2_bpt_mode: V2G
      session_logging: false
      session_logging_path: /tmp
      session_logging_xml: true
      soft_over_current_measurement_noise_A: 0.5
      soft_over_current_timeout_ms: 7000
      soft_over_current_tolerance_percent: 10
      state_F_after_fault_ms: 300
      switch_3ph1ph_cp_state: X1
      switch_3ph1ph_delay_s: 10
      uk_smartcharging_random_delay_at_any_change: true
      uk_smartcharging_random_delay_enable: false
```

```
    uk_smartcharging_random_delay_max_duration: 600
EnergyNode0:
  module: EnergyNode
  connections:
    energy_consumer:
      - module_id: EvseManager0
        implementation_id: energy_grid
  config_module:
    fuse_limit_A: 100
    phase_count: 1
EnergyManager0:
  module: EnergyManager
  connections:
    energy_trunk:
      - module_id: EnergyNode0
        implementation_id: energy_grid
  config_module:
    debug: false
    nominal_ac_voltage: 230
    schedule_interval_duration: 60
    schedule_total_duration: 1
    slice_ampere: 0.5
    slice_watt: 500
    switch_3ph1ph_max_nr_of_switches_per_session: 0
    switch_3ph1ph_power_hysteresis_W: 200
    switch_3ph1ph_switch_limit_stickyness: DontChange
    switch_3ph1ph_time_hysteresis_s: 600
    switch_3ph1ph_while_charging_mode: Never
    update_interval: 1
EvseSlac0:
  module: EvseSlac
  connections: {}
  config_implementation:
    main:
      ac_mode_five_percent: true
      chip_reset_delay_ms: 100
      chip_reset_timeout_ms: 500
      debug_simulate_failed_matching: false
      device: eth1
      do_chip_reset: false
      link_status_detection: false
      link_status_retry_ms: 100
      link_status_timeout_ms: 10000
      number_of_sounds: 10
      publish_mac_on_first_parm_req: false
      publish_mac_on_match_cnf: true
      set_key_timeout_ms: 1000
      sounding_attenuation_adjustment: 0
EvseV2G0:
  module: EvseV2G
  connections:
    security:
      - module_id: EvseSecurity0
        implementation_id: main
  config_module:
    auth_timeout_eim: 300
    auth_timeout_pnc: 55
    device: eth1
    enable_sdp_server: true
    supported_DIN70121: true
    supported_ISO15118_2: false
    terminate_connection_on_failed_response: false
    tls_key_logging: false
    tls_key_logging_path: /tmp
    tls_security: prohibit
    tls_timeout: 15000
    verify_contract_cert_chain: false
EvseSecurity0:
  module: EvseSecurity
```

```
    connections: {}
    config_module:
      csms_ca_bundle: ca/csms/CSMS_ROOT_CA.pem
      csms_leaf_cert_directory: client/csms
      csms_leaf_key_directory: client/csms
      mf_ca_bundle: ca/mf/MF_ROOT_CA.pem
      mo_ca_bundle: ca/mo/MO_ROOT_CA.pem
      private_key_password: ''
      secc_leaf_cert_directory: client/cso
      secc_leaf_key_directory: client/cso
      v2g_ca_bundle: ca/v2g/V2G_ROOT_CA.pem
  DCSupplySimulator0:
    module: DCSupplySimulator
    connections: {}
    config_implementation:
      main:
        bidirectional: false
        max_current: 200
        max_power: 150000
        max_voltage: 900
        min_current: 1
        min_voltage: 200
  IMDSimulator0:
    module: IMDSimulator
    connections: {}
    config_implementation:
      main:
        interval: 1000
        resistance_F_Ohm: 900000
        selftest_success: true
```

The configuration specifies all required modules, their respective parameters, and the connections between them. In the current implementation, the Insulation Monitoring Device (IMD) and DC power supply are simulated.

An additional test configuration for AC charging simulation using the HAT is provided in **`config-raspi-evse-AC.yaml`**.

Example DC EV simulation configuration

The file **`EVerest-2026.02.1/config/config-raspi-ev-DC.yaml`** defines a basic DC EV simulation using the Raspi EV(SE) HAT:

```
settings:
  mqtt_everest_prefix: prefix
active_modules:
  iso15118_ev:
    module: PyEvJosev
    config_module:
      device: eth1
      supported_DIN70121: true
      supported_ISO15118_2: false
      supported_ISO15118_20_DC: false
      tls_active: false
  ev_bsp:
    module: RaspiEvBSP
    config_module:
      i2c_dev: /dev/i2c-1
      i2c_freq: 10000
      i2c_addr: 96
  ev_slac:
```

```
    module: EvSlac
    config_implementation:
      main:
        device: eth1
  ev_manager:
    module: EvManager
    config_module:
      connector_id: 1
      auto_enable: true
      auto_exec: true
      auto_exec_infinite: true
      auto_exec_commands:
wait_for_real_plugin;iso_wait_pwm_is_running;iso_wait_slac_matched;iso_start_v2g_session
DC;iso_wait_pwr_ready;iso_dc_power_on;iso_wait_for_stop 60;iso_wait_v2g_session_stopped;unplug;sleep 5;
      dc_target_current: 5
      dc_target_voltage: 444
    connections:
      ev_board_support:
        - module_id: ev_bsp
          implementation_id: main
      ev:
        - module_id: iso15118_ev
          implementation_id: ev
      slac:
        - module_id: ev_slac
          implementation_id: main
```

The sequence of actions performed during the EV simulation is defined by the **`auto_exec_commands`** configuration parameter.

Currently it runs the following script:

- ➔ wait_for_real_plugin;
- ➔ iso_wait_pwm_is_running;
- ➔ iso_wait_slac_matched;
- ➔ iso_start_v2g_session DC;
- ➔ iso_wait_pwr_ready;
- ➔ iso_dc_power_on;
- ➔ iso_wait_for_stop 60;
- ➔ iso_wait_v2g_session_stopped;
- ➔ unplug;
- ➔ sleep 5;

For available commands and further details see EvManager module.